LogPulse - приём, разбор и статистика сервисных логов

Дисциплина: Внедрение и поддержка компьютерных систем **Форма:** индивидуальное практическое задание для ликвидации академической задолженности **Учебный год:** 2025/2026, 4 курс, группа 9/4-РПО-23/1 **Выполнил:** Гущин Даниил (GitHub: shargabong) **Преподаватель:** Макаров Максим Николаевич **Дата:** 07.11.2026

1. Цель

Сделать LogPulse - систему из HTTP API на FastAPI, воркера и Redis. Она принимает строки логов, проверяет их регулярками, валидные записи складывает в Redis Stream, воркер их читает и считает статистику. Всё поднимается одним docker compose.

2. Что сделано

Парсер и регулярки

Формат строки такой:

```
<UTC ISO-8601 Z> <LEVEL> <service> <IPv4> <METHOD> <path> <status> <N>ms rid=<8 hex>
```

Основной разбор делает одна регулярка с именованными группами (timestamp, level, service, ip, method, path, status, duration_ms, rid), строка проверяется целиком через re.fullmatch, без split. Плюс есть отдельные регулярки: TS_RE, IPV4_RE, RID_RE, PATH_RE и PROBLEM_RE (поиск WARN/ERROR). У каждой свой тест.

Потом поля проверяются типами: timestamp через datetime, IP через ipaddress, код ответа в диапазоне 100-599, длительность 0-600000 мс, level/service/method из списка. Если что-то не так, функция кидает ParseError с причиной, и такая строка в очередь не попадает.

Тестовый файл data/sample.log: 45 корректных строк (три сервиса, три уровня, 12 разных кодов, уникальные rid) и 6 намеренно испорченных.

Таблица 1. Проверка регулярок и ошибок

Пример / тип	Ожидалось	Получилось	Пояснение
INFO auth 192.0.2.10 GET /api/login 200 12ms rid=00000001	Разбор	Разобрана, status=200 (int)	Строка совпала с маской
ERROR orders ... 503 742ms rid=ee44cc22	Разбор	Разобрана, попала в logpulse:slow	742 больше порога 500
2026-13-45T99:99:99Z INFO ...	Ошибка timestamp	timestamp не корректная дата	Маску прошла, datetime не пропустил
INFO auth 999.1.1.1 ...	Ошибка IP	неверный IP: 999.1.1.1	ipaddress не принял октет 999
DEBUG auth ...	Плохой level	строка не соответствует формату	DEBUG нет в маске
ERROR orders ... 999 ...	Плохой статус	код ответа вне 100-599: 999	Маску прошла, диапазон нет
INFO payment ... 200 -5ms	Отрицательная длительность	строка не соответствует формату	-5ms не совпало с \d+ms
WARN orders ... rid=ZZZ12cd3	Плохой rid	строка не соответствует формату	rid не из hex

Пример / тип	Ожидалось	Получилось	Пояснение
Пустая строка (пробелы)	Отказ	пустая строка	Проверка до маски

Redis: очередь и статистика

Очередь сделана через Redis Stream logpulse:events. API кладёт туда события командой XADD. Воркер читает их через XREADGROUP (группа logpulse-workers), обновляет статистику и подтверждает через XACK.

Таблица 2. Структуры Redis

Ключ	Тип	Запись / чтение	Зачем
logpulse:events	Stream	XADD / XREADGROUP, XREVRANGE	Очередь событий
logpulse:stats:level	Hash	HINCRBY / HGETALL	Сколько событий каждого уровня
logpulse:stats:service	Hash	HINCRBY / HGETALL	Сколько событий по сервисам
logpulse:stats:status	Hash	HINCRBY / HGETALL	Сколько ответов по кодам
logpulse:slow	Sorted Set	ZADD / ZREVRANGE	Рейтинг медленных (score = duration_ms, member = rid)

Порог медленного запроса: duration_ms >= 500.

Как понять "принято в очередь" и "обработано воркером": принято - это XLEN у потока, обработано - это total из /stats (сумма счётчиков уровней). После загрузки sample.log XLEN = 45 и total = 45.

FastAPI, Pydantic и ошибки

API на FastAPI: маршруты собраны в APIRouter, модели на Pydantic. Маршруты: /health, /ingest (принимает {"lines": [...]}), /stats и /events?limit=10. Лимиты: от 1 до 100 строк, каждая от 1 до 500 символов. Доступ к Redis идёт через Depends, в тестах он подменяется. Ошибки: 422 на неверный запрос, 503 если Redis недоступен, ответы в одном формате {"error": {"type", "message", "details"}}.

Таблица 3. Проверка API и тестов

Сценарий	Вход	Ожидалось	Получилось
POST /ingest	3 валидные + 2 ошибки	accepted=3, rejected=2	Так и есть, 2 пояснения
POST /ingest	sample.log (45 + 6)	accepted=45, rejected=6	Подтверждено
GET /stats	после воркера	Агрегаты из Redis	total=45, уровни 15/15/15
GET /health	Redis работает	Готовность	200, {"status":"ok","redis":"ok"}
GET /health	Redis остановлен	503	503, быстро
GET /events?limit=1	2 события	1 последнее	count=1, rid=ee44cc22
POST /ingest	101 строка	Отказ по лимиту	422
pytest	тест парсера, IP, top_slow	Пройдены	Пройдены

Docker Compose

Есть Dockerfile, compose.yaml и .env.example. В compose три сервиса: api, worker, redis. Настройки через переменные окружения, внутри контейнеров Redis по имени redis, а не localhost. У redis есть healthcheck, api и worker ждут его готовности. Данные Redis лежат в volume redis-data. HTTP открыт только на 127.0.0.1, порт redis наружу не торчит.

Таблица 4. Журнал инцидента

Шаг	Действие	Ожидалось	Факт	Доказательство
1	docker compose up --build -d	Три контейнера готовы	redis healthy, api и worker подняты	вывод docker compose ps
2	Загрузить sample.log	Статистика построена	accepted=45, rejected=6, total=45	ответ /ingest и /stats
3	docker compose restart redis	Краткая недоступность	Воркер ждёт и повторяет попытку	строки лога воркера о повторе
4	Дождаться восстановления	Сервис снова работает	/health = 200, поток дообработан	ответ /health, XINFO GROUPS (lag=0)
5	Повторить /stats	Данные на месте	total=45, не изменилось	ответ /stats после restart

Отдельно проверен случай полной остановки redis: `/health`, `/stats` и `/ingest` возвращают 503, после `docker compose start redis` доступность и статистика восстанавливаются из volume.

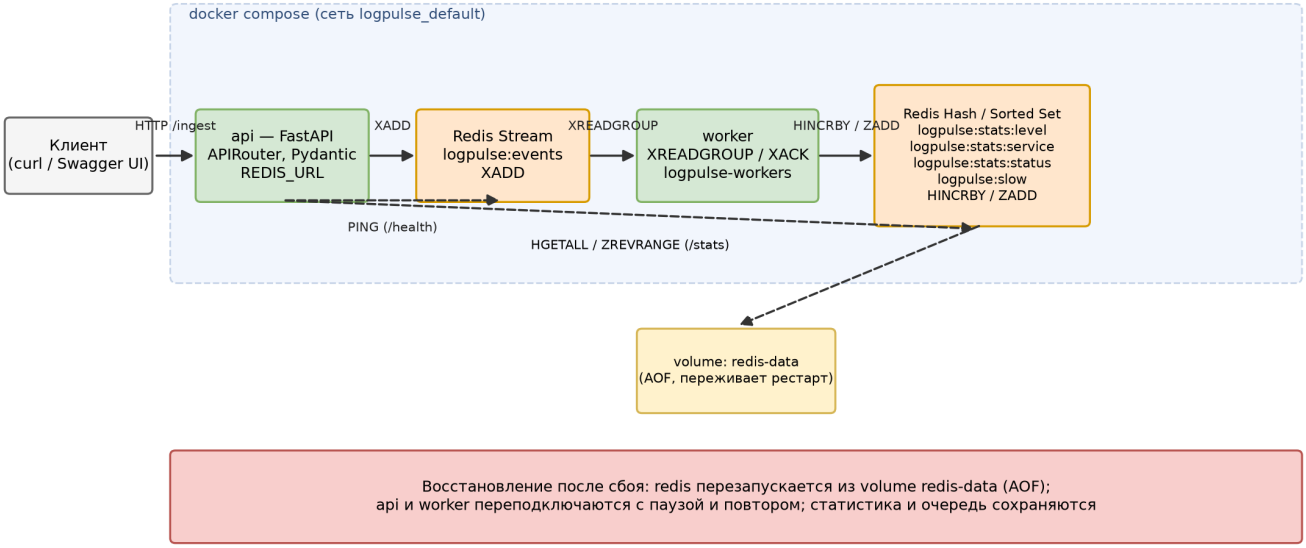
Git и тесты

Тестов 35 (нужно было минимум 12). Покрыты: корректная строка, битый timestamp, плохой IP, неподдерживаемый уровень, плохой статус, отрицательная и большая длительность, плохой rid, пустая строка, накопление счётчиков, рейтинг медленных, лимиты /events и /ingest, /health без Redis (503).

Репозиторий ведётся с начала: несколько коммитов, две ветки (feature/parser и feature/redis) со слиянием в main, .gitignore. История выгружена в history.bundle.

```
$ git log --graph --oneline --all --decorate
* 2d3f7a5 (HEAD -> main) refactor: упростил код - синхронный redis, меньше файлов
* 913cf66 docs: актуализировал в отчёте вывод git log
* d341d0f chore: .dockerignore для сборки образов
* 04da591 docs: README, архитектурная схема, отчёт и утилиты
*   4783566 merge: Redis, воркер и API (feature/redis)
|\
| * 10ebfe6 (feature/redis) feat(worker): чтение Redis Streams и тесты API и статистики
| * b981f1c feat(api): роутеры health/ingest/stats/events и единый формат ошибок
| * b0b8cea feat(redis): Pydantic-модели и слой статистики Redis
* |   b2e0986 merge: парсер и данные (feature/parser)
|\ \
| |/
|/|
| * 607b20c (feature/parser) test(parser): синтетический data/sample.log на 45 + 6 строк
| * 5defeda feat(parser): регулярная маска, проверка полей и тесты
|/
* c61fc53 chore: каркас LogPulse - окружение, Docker и конфиг
```

3. Архитектура



Путь события: Клиент -> API -> Redis Stream -> worker -> Redis Hash/Sorted Set. Отдельной стрелкой API читает статистику из Redis. На схеме также контейнеры, переменные окружения (REDIS_URL), volume redis-data и восстановление после перезапуска. Редактируемый исходник: architecture.drawio.

4. Примеры логов и ответов


```
{
  "accepted": 3,
  "rejected": 2,
  "errors": [
    {"line": 4, "reason": "код ответа вне 100-599: 999"},
    {"line": 5, "reason": "неверный IP: 999.1.1.1"}
  ]
}
```

```
{
  "total": 45,
  "by_level": {"INFO": 15, "WARN": 15, "ERROR": 15},
  "by_service": {"auth": 15, "orders": 15, "payment": 15},
  "by_status": {"200": 4, "201": 4, "204": 4, "301": 4, "400": 4, "401": 4,
                "403": 4, "404": 4, "429": 4, "500": 3, "502": 3, "503": 3},
  "top_slow": [
    {"rid": "0000002c", "duration_ms": 1500},
    {"rid": "00000021", "duration_ms": 1500},
    {"rid": "00000016", "duration_ms": 1500},
    {"rid": "0000000b", "duration_ms": 1500},
    {"rid": "0000002b", "duration_ms": 980}
  ]
}
```


5. Скриншоты

ЗАГЛУШКА

Swagger UI (/docs)

Замените на собственный скриншот с этим именем файла

ЗАГЛУШКА

Тестовый запрос POST /ingest из /docs

Замените на собственный скриншот с этим именем файла

ЗАГЛУШКА

redis-cli: XINFO STREAM / HGETALL / ZREVRANGE

Замените на собственный скриншот с этим именем файла

ЗАГЛУШКА

docker compose ps и логи

Замените на собственный скриншот с этим именем файла





6. Инцидент с Redis

Проверял сценарий из задания: запуск, загрузка sample.log, docker compose restart redis, ожидание и повторный /health и /stats. Дополнительно проверял полную остановку redis.

Сначала наткнулся на проблему: когда Redis лежит, /health не отвечал 503, а просто висел. Оказалось, у клиента redis не был задан таймаут сокета, поэтому запрос ждал ответа бесконечно. С точки зрения оператора это выглядело как зависание. Добавил socket_connect_timeout и socket_timeout по 2 секунды, теперь /health и /stats быстро отдают 503.

Данные при перезапуске не теряются. Redis пишет с --appendonly yes в volume redis-data. После restart и даже после stop/start total=45, счётчики и top_slow остались на месте. Команду down -v для такой проверки использовать нельзя, она удаляет volume.

При недоступном Redis приложение не падает: uvicorn продолжает работать, все маршруты отдают 503 в общем формате, после возврата Redis снова 200. Воркер тоже не падает: пишет в лог, что нет связи, ждёт 2 секунды и пробует снова, а после восстановления продолжает читать с того же места группы.

Ещё замечание про полную остановку redis (не restart, а stop): из DNS пропадает имя сервиса, и в логе воркера появляется обычная запись вида "нет связи с Redis: Error -2 connecting to redis:6379. Name or service not known". Воркер её ловит, ждёт 2 секунды и пробует снова, на работу это не влияет. Для штатного restart такого нет, потому что имя сервиса остаётся.

Вывод: сервис нормально переживает недоступность Redis, данные сохраняются, воркер переподключается. Найденный баг с зависанием /health был реальный, его исправил.

7. Комплектность

Таблица 5. Контроль комплектности

Артефакт	Требование	Файл	Проверено
FastAPI-проект	FastAPI + парсер + worker	app/, worker.py	Да
Redis	Stream + Hash + Sorted Set	app/stats.py, app/config.py	Да
Docker Compose	3 сервиса + volume	compose.yaml, Dockerfile	Да
Тесты	>= 12 pytest	tests/ (35 тестов)	Да
Данные	>= 40 логов и >= 5 битых	data/sample.log (45 + 6)	Да
Git	>= 6 коммитов, 2 ветки	history.bundle	Да
Отчёт	PDF + 5 таблиц + скриншоты	report.pdf	Да
Схема	.drawio	architecture.drawio	Да
Домашние работы	Все прошлые ДЗ	сдаются отдельно	Да

8. Ограничения

- Если сообщение придёт воркеру дважды, счётчики могут увеличиться дважды. В задании такие дубликаты описаны как допустимое ограничение.
- Воркер один, шардирования нет.